环境准备

系统要求

1. 一台或多台运行兼容 deb/rpm 的 Linux 操作系统的计算机；例如：Ubuntu 或 CentOS。
2. 每台机器 2 GB 以上的内存，内存不足时应用会受限制。
3. 用作控制平面节点的计算机上至少有2个 CPU。
4. 集群中所有计算机之间具有完全的网络连接。你可以使用公共网络或专用网络。

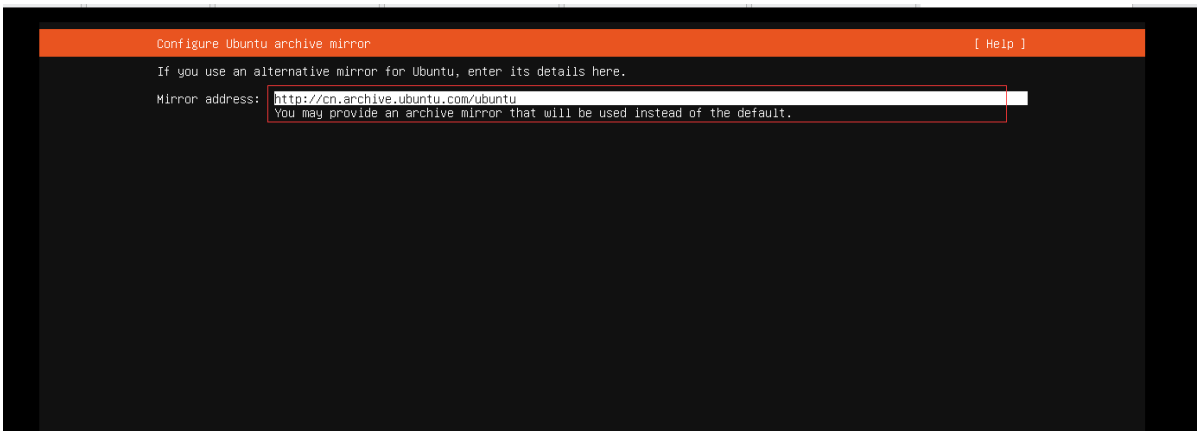
主机环境

1. 宿主机环境 ubuntu-20.04.4-live-server-amd64，下载地址：

https://mirrors.aliyun.com/ubuntu-releases/20.04.4/ubuntu-20.04.4-live-server-amd64.iso

2. apt 包管理器，镜像源修改:

将 http://cn.archive.ubuntu.com/ubuntu 改为 https://mirrors.tuna.tsinghua.edu.cn/ubuntu
两种修改方式：
第一种：在安装虚拟机时，修改图下图：



第二种：已经安装好系统的情况下，修改 /etc/apt/sources.list 将对应的地址替换，替换完成后执行 apt-get update。注意：先备份/etc/apt/sourse.list文件

3. docker版本 24.0.5

docker容器运行时安装

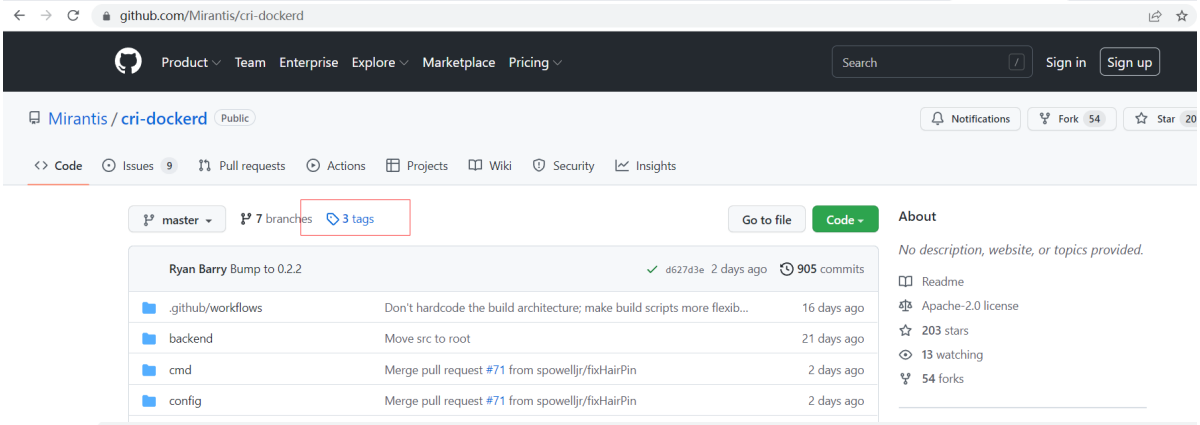
cri-dockerd安装

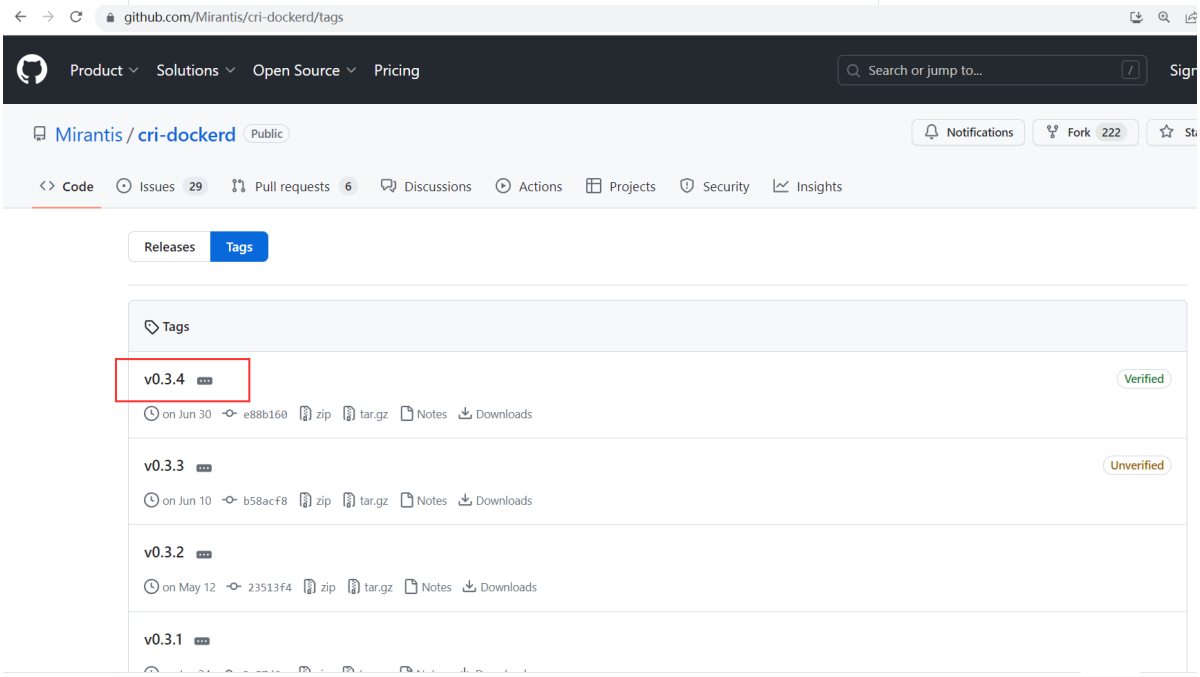
源码地址：https://github.com/Mirantis/cri-dockerd.git

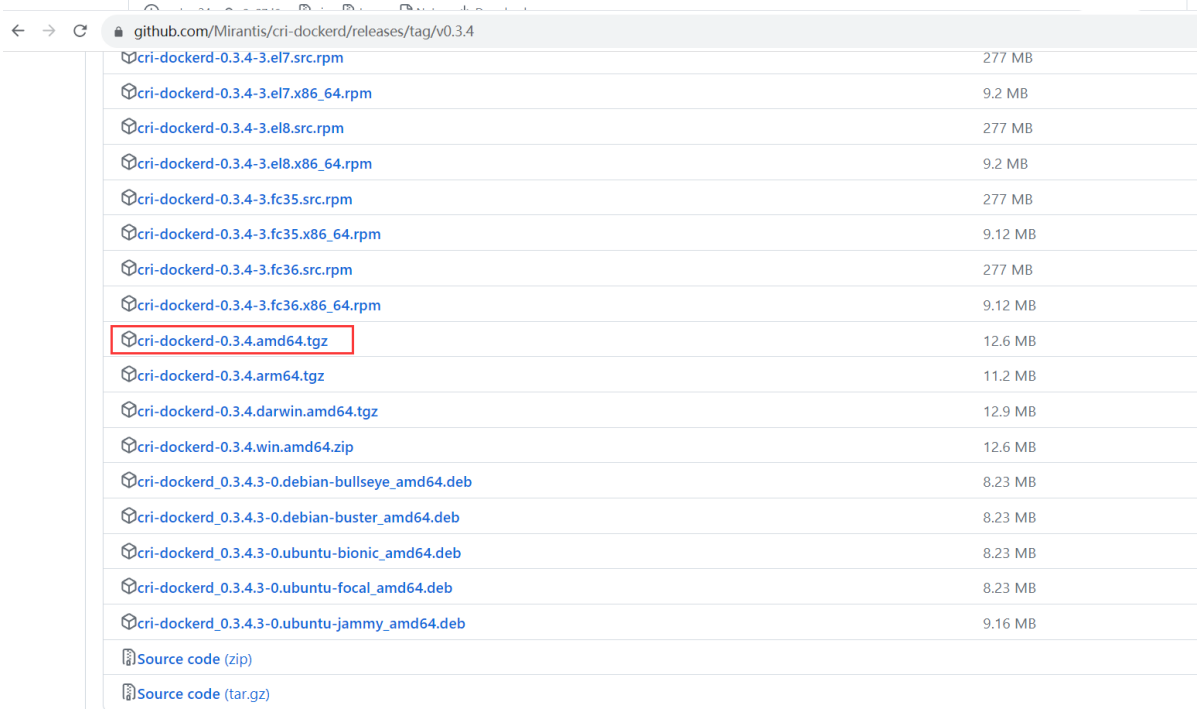
1. 查看内核信息

```
cat /proc/version
```

2. 如何找到下载地址：







3. 根据内核信息选择对应的文件下载地址：

https://github.com/Mirantis/cri-dockerd/releases/download/v0.3.4/cri-dockerd-0.3.4.amd64.tgz

4. 下载并安装

```
# 下载
wget https://github.com/Mirantis/cri-dockerd/releases/download/v0.3.4/cri-dockerd-0.3.4.amd64.tgz
# 解压到/tmp目录
tar -zxvf cri-dockerd-0.3.4.amd64.tgz -C /tmp
# 安装
sudo cp /tmp/cri-dockerd/cri-dockerd /usr/bin/
```

cri-dockerd服务配置和启动

参考：https://github.com/Mirantis/cri-dockerd/tree/master/packaging/systemd

```
# 获取源码，如果已经下载了无需重复下载
git clone https://github.com/Mirantis/cri-dockerd.git
# 进入源码目录
cd cri-dockerd
# 切换到对应tag
git checkout v0.3.4
# 将源中systemd配置文件拷贝到相应目录
sudo cp packaging/systemd/*  /etc/systemd/system/
```

1. 文件 /etc/systemd/system/cri-docker.socket 内容无需修改

```
[Unit]
Description=CRI Docker Socket for the API
PartOf=cri-docker.service

[Socket]
ListenStream=%t/cri-dockerd.sock
SocketMode=0660
SocketUser=root
SocketGroup=docker

[Install]
WantedBy=sockets.target
```

2. 文件 /etc/systemd/system/cri-docker.service Service.ExecStart 字段增加 --network-plugin 与 --pod-infra-container-image 选项。--pod-infra-container-image 镜像tag需要根据具体k8s安装版本指定

```
[Unit]
Description=CRI Interface for Docker Application Container Engine
Documentation=https://docs.mirantis.com
After=network-online.target firewalld.service docker.service
Wants=network-online.target
Requires=cri-docker.socket

[Service]
Type=notify
ExecStart=/usr/bin/cri-dockerd --container-runtime-endpoint fd:// --network-plugin=cni \
          --pod-infra-container-image=registry.aliyuncs.com/google_containers/pause:3.9
```

```
ExecReload=/bin/kill -s HUP $MAINPID
TimeoutSec=0
RestartSec=2
Restart=always

# Note that StartLimit* options were moved from "Service" to "Unit" in systemd
229.
# Both the old, and new location are accepted by systemd 229 and up, so using the
old location
# to make them work for either version of systemd.
StartLimitBurst=3

# Note that StartLimitInterval was renamed to StartLimitIntervalSec in systemd
230.
# Both the old, and new name are accepted by systemd 230 and up, so using the old
name to make
# this option work for either version of systemd.
StartLimitInterval=60s

# Having non-zero Limit*s causes performance problems due to accounting overhead
# in the kernel. We recommend using cgroups to do container-local accounting.
LimitNOFILE=infinity
LimitNPROC=infinity
LimitCORE=infinity

# Comment TasksMax if your systemd version does not support it.
# Only systemd 226 and above support this option.
TasksMax=infinity
Delegate=yes
KillMode=process

[Install]
WantedBy=multi-user.target
```

3. 启动服务

```
# 重新加载配置
sudo systemctl daemon-reload
# 设置为开机自启动
sudo systemctl enable cri-docker
# 启动服务
sudo systemctl start cri-docker
# 检查服务状态
sudo systemctl status cri-docker
```

```
nick@worker2:~$ sudo systemctl status cri-docker
● cri-docker.service - CRI Interface for Docker Application Container Engine
     Loaded: loaded (/etc/systemd/system/cri-docker.service; enabled; vendor preset: enabled)
     Active: active (running) since Fri 2023-08-04 12:30:55 UTC; 18s ago
TriggeredBy: ● cri-docker.socket
       Docs: https://docs.mirantis.com
   Main PID: 10056 (cri-dockerd)
      Tasks: 10
     Memory: 11.4M
     CGroup: /system.slice/cri-docker.service
             └─10056 /usr/bin/cri-dockerd --container-runtime-endpoint fd:// --network-plugin=cni --pod-infra-container-image=registry.aliyuncs.com/google_containers/pause:3.9

Aug 04 12:30:55 worker2 cri-dockerd[10056]: time="2023-08-04T12:30:55Z" level=info msg="Start docker client with request timeout 0s"
Aug 04 12:30:55 worker2 cri-dockerd[10056]: time="2023-08-04T12:30:55Z" level=info msg="Hairpin mode is set to none"
Aug 04 12:30:55 worker2 cri-dockerd[10056]: time="2023-08-04T12:30:55Z" level=info msg="Loaded network plugin cni"
Aug 04 12:30:55 worker2 cri-dockerd[10056]: time="2023-08-04T12:30:55Z" level=info msg="Docker cri networking managed by network plugin cni"
Aug 04 12:30:55 worker2 cri-dockerd[10056]: time="2023-08-04T12:30:55Z" level=info msg="Docker Info: &{ID:6353168f-2533-494a-b9a9-8c02e939214a Containers:0 ContainersRunning:0 C
Aug 04 12:30:55 worker2 cri-dockerd[10056]: time="2023-08-04T12:30:55Z" level=info msg="Setting cgroupDriver cgroupfs"
Aug 04 12:30:55 worker2 cri-dockerd[10056]: time="2023-08-04T12:30:55Z" level=info msg="Docker cri received runtime config &RuntimeConfig{NetworkConfig:&NetworkConfig{PodCidr:,}}
Aug 04 12:30:55 worker2 cri-dockerd[10056]: time="2023-08-04T12:30:55Z" level=info msg="Starting the GRPC backend for the Docker CRI interface."
Aug 04 12:30:55 worker2 cri-dockerd[10056]: time="2023-08-04T12:30:55Z" level=info msg="Start cri-dockerd grpc backend"
Aug 04 12:30:55 worker2 systemd[1]: Started CRI Interface for Docker Application Container Engine.
lines 1-21/21 (END)
```

kubeadm和相关工具

kubeadm是Kubernetes从1.4版本开始引入的命令行工具，致力于简化集群的安装过程，并解决Kubernetes集群高可用问题。

1. 源码地址：https://github.com/kubernetes/kubeadm
2. 安装说明文档地址：

https://kubernetes.io/docs/setup/production-environment/tools/kubeadm/install-kubeadm/

安装kubeadm、kubelet、kubectl

1. 更新包管理器索引，并安装相关软件支持HTTPS访问

```
sudo apt-get update
sudo apt-get install -y apt-transport-https ca-certificates curl
```

2. 下载gpg秘钥

```
# 请使用此方式获取gpg key,需要下载之后，在本地用gpg加密
curl -fsSL https://mirrors.aliyun.com/kubernetes/apt/doc/apt-key.gpg | sudo gpg --dearmor -o /usr/share/keyrings/kubernetes-archive-keyring.gpg
```

3. 设置Kubernetes镜像源

```
echo "deb [signed-by=/usr/share/keyrings/kubernetes-archive-keyring.gpg] https://mirrors.aliyun.com/kubernetes/apt/ kubernetes-xenial main" | sudo tee /etc/apt/sources.list.d/kubernetes.list
```

4. 更新apt软件索引，并查看相关软件的可用版本

```
sudo apt-get update
apt-cache madison kubelet kubeadm kubectl
```

5. 安装特定版本

```
sudo apt-get install -y kubelet=<VERSION_STRING> kubeadm=<VERSION_STRING> kubectl=<VERSION_STRING>
例如:
sudo apt-get install -y kubelet=1.27.4-00 kubeadm=1.27.4-00 kubectl=1.27.4-00
```

6. 安装最新版本

```
sudo apt-get install -y kubelet kubeadm kubectl
```

7. 锁定软件版本

```
# 防止自动升级更新
sudo apt-mark hold kubelet kubeadm kubectl
```

8. 检查kubelet状态

```
systemctl status kubelet
```

注意：kubelet在刚安装完成时，会处于一个自动启动状态，每10s启动一次，在没有完成初始化之前它一致处于这种状态，所以不要纠结于kubelet安装之后没有启动。

卸载

```
sudo apt-get remove kubelet kubectl kubeadm
```

主机环境调整

修改节点上Docker 的cgroup driver为systemd

由于Kubernetes中kubelet组件默认使用的cgroupdriver 为 “systemd”,所以我们将docker的cgroupdriver也修改为 “systemd”
参考文档：
https://kubernetes.io/docs/tasks/administer-cluster/kubeadm/configure-cgroup-driver/
https://kubernetes.io/docs/setup/production-environment/container-runtimes/

```
# 打开docker配置文件进行修改，该文件默认情况下不存在，可以新建一个，不同操作系统的文件位置不一样
sudo vim /etc/docker/daemon.json
# 增加配置内容
"exec-opts": [
    "native.cgroupdriver=systemd"
  ]

# 重新加载配置并重启服务
sudo systemctl daemon-reload && sudo systemctl restart docker
```

新增配置文件内容如下：

```
{
  "exec-opts": [
    "native.cgroupdriver=systemd"
  ]
}
```

关闭防火墙

```
sudo systemctl stop firewalld
sudo systemctl disable firewalld
```

禁用Selinux

selinux，这个是用来加强安全性的一个组件，但非常容易出错且难以定位，一般上来装完系统就先给禁用了

```
# 查看 selinux 状态
sudo apt install selinux-utils
getenforce

# 禁用
sudo setenforce 0
```

禁用swap

```
# 查看交换区
free
# 禁用交换区
sudo swapoff -a
# 打开文件注释交换区定义
/etc/fstab
```

```
# /etc/fstab: static file system information.
#
# Use 'blkid' to print the universally unique identifier for a
# device; this may be used with UUID= as a more robust way to name devices
# that works even if disks are added and removed. See fstab(5).
#
# <file system> <mount point>   <type>  <options>       <dump>  <pass>
# / was on /dev/ubuntu-vg/ubuntu-lv during curtin installation
/dev/disk/by-id/dm-uuid-LVM-dgookVCI706qCKTNISRDTCiz7U5VobDdYRVceTpMVwZeKfjq45sNLcI3vkE0fAIs / ext4 defaults 0 1
# /boot was on /dev/sda2 during curtin installation
/dev/disk/by-uuid/601bb189-32a5-46a3-8253-b71f40c37b4b /boot ext4 defaults 0 1
/swap.img       none    swap    sw      0       0
~
~
~
~
```

1. 查看交换区是否关闭

```
free
```

```
nick@host6:~$ free
              total        used        free      shared  buff/cache   available
Mem:        8142008      327896     3655476        1368     4158636     7508056
Swap:             0           0           0
nick@host6:~$
```

修改主机名

1. /etc/hosts文件增加主机名与本机ip映射

```
127.0.0.1 localhost
127.0.1.1 host6
127.0.1.1 k8s-master1
```

2. 修改系统主机名

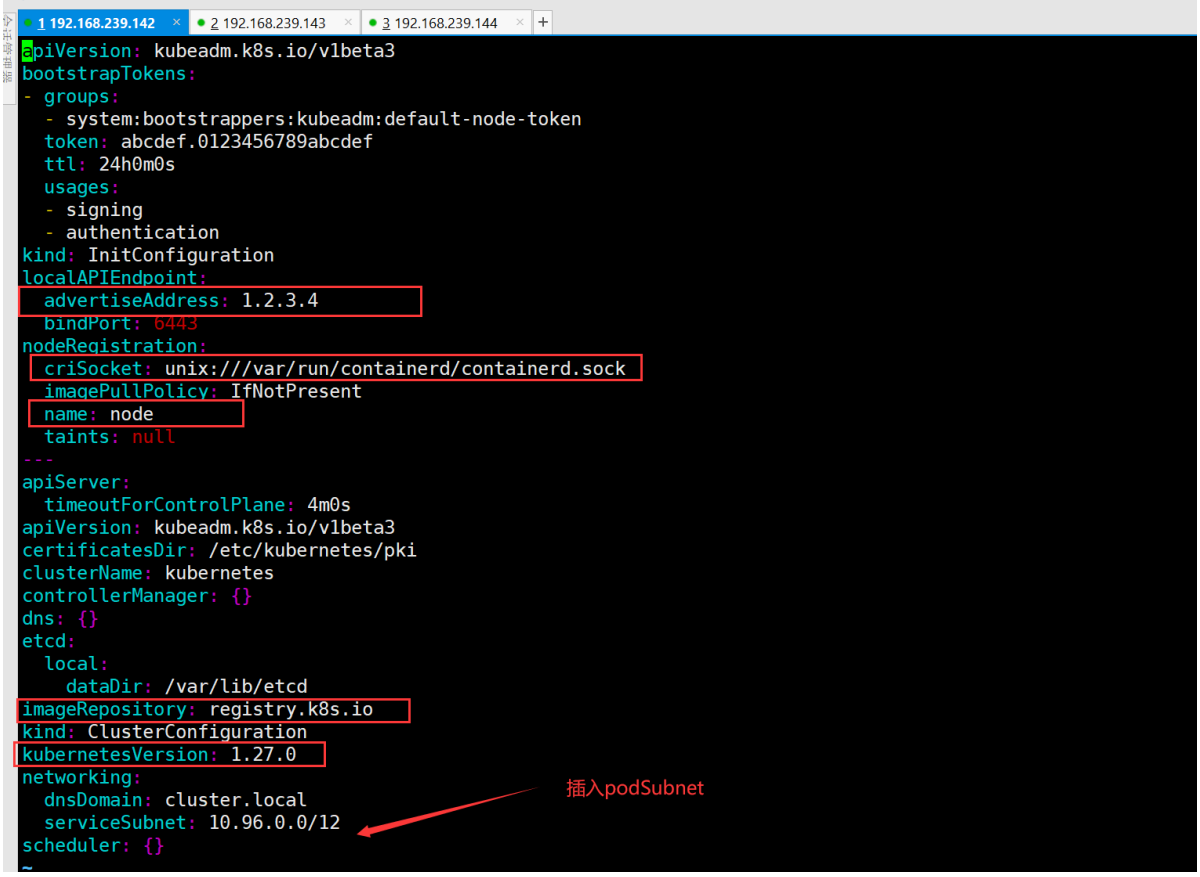
```
# 修改主机名
sudo hostnamectl set-hostname k8s-master1
# 查看主机名
hostname
```

初始化master

生成默认配置文件

```
kubeadm config print init-defaults > init.default.yaml
```

修改配置文件




```
# 修改地址 节点IP地址
localAPIEndpoint.advertiseAddress: 192.168.239.142
# 修改套接字
nodeRegistration.criSocket: unix:///var/run/cri-dockerd.sock
# 修改节点名称
nodeRegistration.name: master1
# 修改镜像仓库地址为国内开源镜像库
imageRepository: registry.aliyuncs.com/google_containers
# 修改版本号
kubernetesVersion: 1.27.4
# 增加podSubnet，由于后续会安装flannel 网络插件，该插件必须在集群初始化时指定pod地址
# 10.244.0.0/16 为flannel组件podSubnet默认值，集群配置与网络组件中的配置需保持一致
networking.podSubnet: 10.244.0.0/16
```

拉取相关镜像

```
sudo kubeadm config images pull --config=init.default.yaml
```

```
nick@master1:~$ sudo kubeadm config images pull --config=init.default.yaml
[config/images] Pulled registry.aliyuncs.com/google_containers/kube-apiserver:v1.27.4
[config/images] Pulled registry.aliyuncs.com/google_containers/kube-controller-manager:v1.27.4
[config/images] Pulled registry.aliyuncs.com/google_containers/kube-scheduler:v1.27.4
[config/images] Pulled registry.aliyuncs.com/google_containers/kube-proxy:v1.27.4
[config/images] Pulled registry.aliyuncs.com/google_containers/pause:3.9
[config/images] Pulled registry.aliyuncs.com/google_containers/etcd:3.5.7-0
[config/images] Pulled registry.aliyuncs.com/google_containers/coredns:v1.10.1
nick@master1:~$
```

初始化集群

```
sudo kubeadm init --config=init.default.yaml
```

初始化成功之后，返回以下内容，按提示执行即可

```
Your Kubernetes control-plane has initialized successfully!

To start using your cluster, you need to run the following as a regular user:

  mkdir -p $HOME/.kube
  sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
  sudo chown $(id -u):$(id -g) $HOME/.kube/config

Alternatively, if you are the root user, you can run:

  export KUBECONFIG=/etc/kubernetes/admin.conf

You should now deploy a pod network to the cluster.
Run "kubectl apply -f [podnetwork].yaml" with one of the options listed at:
  https://kubernetes.io/docs/concepts/cluster-administration/addons/

Then you can join any number of worker nodes by running the following on each as root:

kubeadm join 192.168.239.146:6443 --token abcdef.0123456789abcdef \
        --discovery-token-ca-cert-hash sha256:bac9b53b18b0bfcb3b923f970771e48d489ed7c77625c8fa3785608c6f191a79
```

当前用户为普通用户，请执行以下指令：

```
mkdir -p $HOME/.kube
sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
sudo chown $(id -u):$(id -g) $HOME/.kube/config
```

当前用户为root用户，请配置环境变量：

```
# /etc/profile 末尾添加环境变量
export KUBECONFIG=/etc/kubernetes/admin.conf

# 执行命令，立即生效
source /etc/profile
```

注意：以上两种情况二选一
添加节点到集群命令，注意：需要指定 --cri-socket unix:///var/run/cri-dockerd.sock

```
# 添加节点到集群指令
sudo kueadm join 192.168.239.142:6443 --token abcdef.0123456789abcdef \
    --discovery-token-ca-cert-hash
sha256:4a36128e4f3a58cb7b7a4f34ef27bbe14e31b7357ae65c4ebc5f8e5699925b26
# 如果过期了，在master节点上执行以下命令可重新获得
sudo kubeadm token create --print-join-command
```

查看集群状态

```
# 查看所有pod
sudo kubectl get pods --all-namespaces
# 查看节点
sudo kubectl get node
# 查看所有组件
sudo kubectl get cs
```

添加pod网络组件

文档

网络组件文档：https://kubernetes.io/docs/concepts/cluster-administration/addons/
flannel组件：https://github.com/flannel-io/flannel#deploying-flannel-manually

flannel安装

```
kubectl apply -f https://github.com/flannel-io/flannel/releases/latest/download/kube-flannel.yml
```

```
nick@master1:~$ kubectl apply -f https://github.com/flannel-io/flannel/releases/latest/download/kube-flannel.yml
namespace/kube-flannel created
serviceaccount/flannel created
clusterrole.rbac.authorization.k8s.io/flannel created
clusterrolebinding.rbac.authorization.k8s.io/flannel created
configmap/kube-flannel-cfg created
daemonset.apps/kube-flannel-ds created
nick@master1:~$ kubectl get pod -A
NAMESPACE      NAME                                 READY   STATUS              RESTARTS   AGE
kube-flannel   kube-flannel-ds-25nbd                0/1     Init:1/2            0          75s
kube-flannel   kube-flannel-ds-6blbs                0/1     Init:0/2            0          75s
kube-flannel   kube-flannel-ds-sc24q                0/1     Init:0/2            0          75s
kube-system    coredns-7bdc4cb885-tvbfq             0/1     ContainerCreating   0          8m20s
kube-system    coredns-7bdc4cb885-wjvrl             0/1     ContainerCreating   0          8m20s
kube-system    etcd-master1                         1/1     Running             0          8m32s
kube-system    kube-apiserver-master1               1/1     Running             0          8m34s
kube-system    kube-controller-manager-master1      1/1     Running             0          8m32s
kube-system    kube-proxy-c9jhh                     0/1     ContainerCreating   0          4m27s
kube-system    kube-proxy-cbdgm                     1/1     Running             0          8m20s
kube-system    kube-proxy-vshg7                     0/1     ContainerCreating   0          4m39s
kube-system    kube-scheduler-master1               1/1     Running             0          8m32s
nick@master1:~$
```

kube-flannel.yml
集群初始化后会存在部分pod处于未启动状态，同时部分节点也处于notready状态；flannel、proxy、coredns 的启动需要相应的镜像以及pause镜像，coredns还需要CNI的具体实现即网络的安装

开启kube-proxy的ipvs模式

```
# 修改mod
kubectl edit -n kube-system cm kube-proxy
修改：mode: "ipvs"

# 重启kube-proxy 守护进程
kubectl rollout restart -n kube-system daemonset kube-proxy
```

查看集群状态：kubectl get pod --all-namespaces 或 kubectl get pod -A

```
nick@k8s-master1:~/work$ kubectl get pod -A
NAMESPACE      NAME                                 READY   STATUS    RESTARTS   AGE
kube-flannel   kube-flannel-ds-4qmm6                1/1     Running   0          15m
kube-flannel   kube-flannel-ds-9c7v4                1/1     Running   0          15m
kube-flannel   kube-flannel-ds-dtqkx                1/1     Running   0          15m
kube-system    coredns-c676cc86f-6sxmw              1/1     Running   0          22m
kube-system    coredns-c676cc86f-wns8d              1/1     Running   0          22m
kube-system    etcd-k8s-master1                     1/1     Running   0          22m
kube-system    kube-apiserver-k8s-master1           1/1     Running   0          22m
kube-system    kube-controller-manager-k8s-master1  1/1     Running   0          22m
kube-system    kube-proxy-7mkqg                     1/1     Running   0          16m
kube-system    kube-proxy-jz4mt                     1/1     Running   0          16m
kube-system    kube-proxy-krw5l                     1/1     Running   0          22m
kube-system    kube-scheduler-k8s-master1           1/1     Running   0          20m
```

到此，完成master节点的初始化

若有pod状态不正常，可通过查看日志获取错误信息

```
kubectl -n kube-system logs <pod>
```

Node节点初始化

环境安装

1. 安装docker
2. 安装cri-dockerd 运行时，并启动服务
3. kubeadm、kubelet、kubectl等工具安装

节点环境修改

1. 修改docker的Cgroup Driver
2. 关闭防火墙
3. 禁用selinux
4. 禁用swap
5. 调整主机名（若需要）

将节点添加到集群

```
# 在master节点上执行以下命令可获得
sudo kubeadm token create --print-join-command
# 通过以上命令获得加入到集群的命令后，
# 在命令上附加参数 --cri-socket unix:///var/run/cri-dockerd.sock
```

重置节点

```
sudo kubeadm reset
或
sudo kubeadm reset --cri-socket unix:///var/run/cri-dockerd.sock
```

```
sudo rm -rf /var/lib/kubelet /var/lib/dockershim /var/run/kubernetes /var/lib/cni
/etc/cni/net.d $HOME/.kube/config
```

```
sudo ipvsadm --clear
```

```
# 删除网络
sudo ifconfig cni0 down && sudo ip link delete cni0
```